SECTION 2 · SUBSECTION 2.1
TOPIC CATALOG ADMINISTRATION (CRUD)

2.1.1 INTRODUCTION.

The /topics module is read-only in the frontend, although the backend already provides POST, PUT and DELETE on /api/topics, soft delete, a hierarchical tree and the classifier cache. This plan lets a user administer the catalog from the UI without touching the database: list as a tree, create with an optional parent, edit name and description, activate or deactivate, and delete (figure 2.1-1).

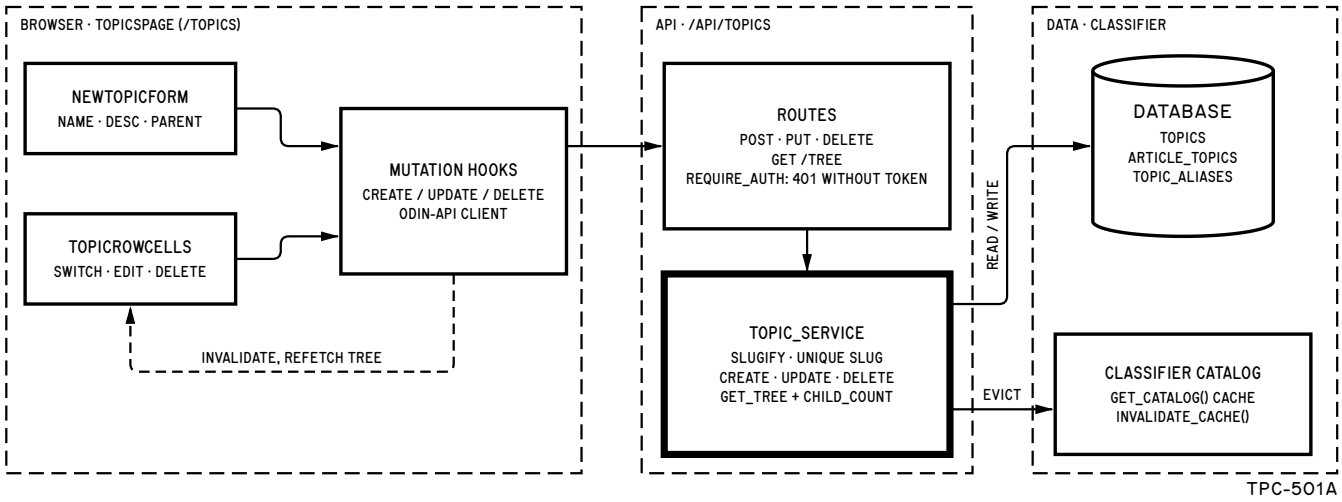


Figure 2.1-1. Topic CRUD Functional Block Diagram

2.1.2 FUNCTIONAL DESCRIPTION.

Work runs in four packages after four baseline prerequisites: backend (13 tasks), frontend contracts (4), components (9) and integration (5), as in figure 2.1-2. The client has not sent its topic list, so the CRUD assumes no structure: the parent is optional and the catalog works flat or hierarchical. Create and edit are inline, as in AliasManager.tsx, because the repo has no form modal.

2.1.2.1 Task Rule.

Every task writes its test, watches it fail for the stated reason, implements the minimum, goes green and ticks its checkbox (figure 2.1-10). Characterization tests are born green and say so.

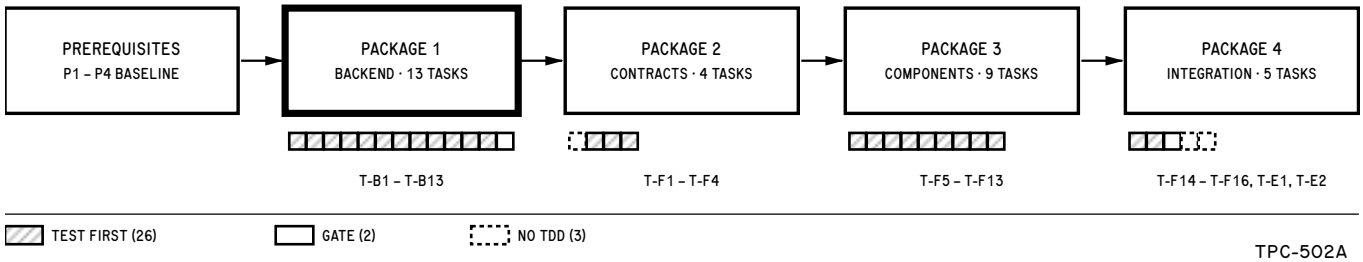


Figure 2.1-2. Package Order and Task Tally

2.1.3 CHANGES TO THE PLAN.

Table 2.1-1. Changes Against the Original Plan

Ref	Change
C1	child_count on TopicNode; DELETE stays 204
C2	T-B9: no orphan aliases after a hard delete
C3	T-B12: blank description stored as NULL
C4	T-B11: cache invalidation spy per mutation
C5	slug split in 4 tasks; one behaviour each
C6	explicit 422 for names without letters
C7	baseline prerequisites P1–P3 made gates
C8	four packages with checklists; order kept

CAUTION

Make no commits. Never call the Gemini API (use LocalAnalyzer). Leave the files of the redesign untouched, except those the plan names.

BASELINE P1–P4 **DEV**

1 BACKEND - run the P1 suites
  √ green, else STOP and tell user
2 VITEST - run full suite (P2)
  √ note failures already red
3 GIT - status, diff --stat (P3)
  √ 7 files clean, else warn
4 READ - the 6 files named in P4

TPC

SECTION 2 · SUBSECTION 2.1
TOPIC CATALOG ADMINISTRATION (CRUD)

2.1.4 PACKAGE 1 · SLUG AND DESCRIPTION.

A user types only the name of a topic. TopicPayload.slug becomes optional (max 160 characters) and create_topic resolves it as in figure 2.1-3: an explicit slug is kept, otherwise the slug is derived from the name with slugify (text_norm.py). A blank description is stored as NULL, never as an empty string. After each successful write the classifier catalog cache is invalidated (figure 2.1-4).

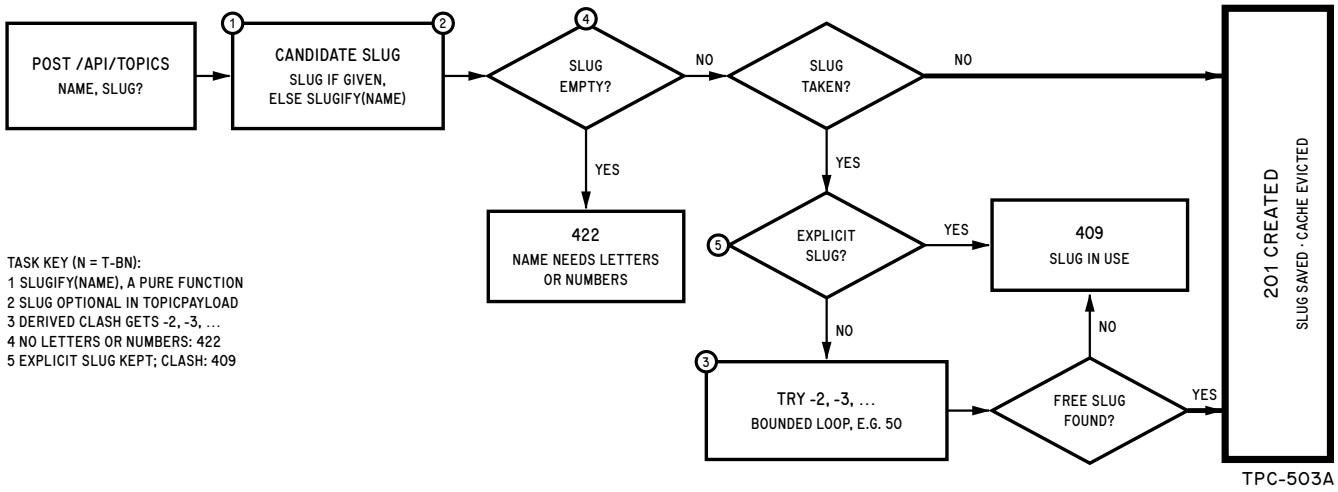


Figure 2.1-3. Slug Resolution in create_topic

2.1.4.1 Slug Rule.

slugify strips accents, lowercases, replaces each run of characters outside a–z and 0–9 with one hyphen, and trims hyphens at both ends. A name that yields an empty slug is answered with 422.

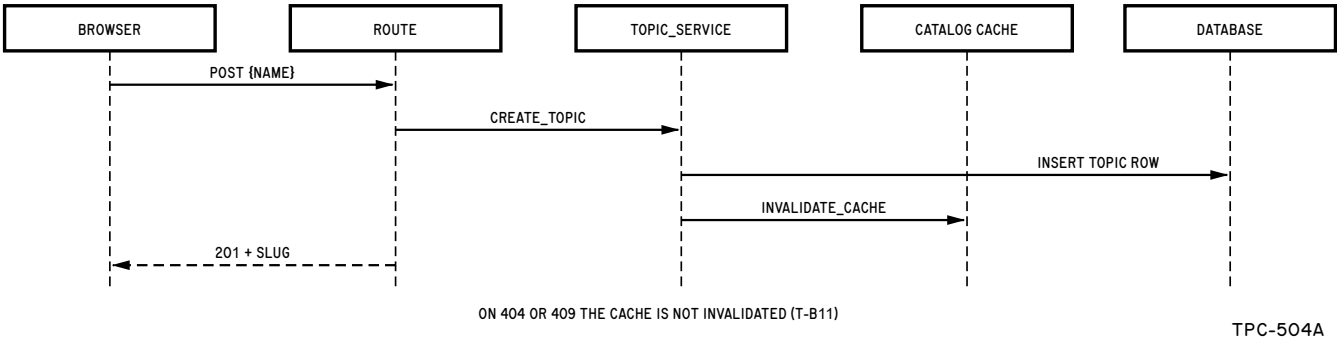


Figure 2.1-4. Successful Create: Message Sequence

2.1.5 SLUG AND DESCRIPTION CASES.

Table 2.1-2. Cases Fixed by Tasks T-B1 to T-B5 and T-B12

Request	Result
"Política y Gobierno"	201 politica-y-gobierno
"a-b" through slugify	a-b
"Salud" under 2nd parent	201 salud-2
"Salud", third time	201 salud-3
"***", no slug	422, no row created
slug "mi-slug"	201 mi-slug
slug "mi-slug" again	409, no auto suffix
description ""	NULL stored

CAUTION

The slug column is not UNIQUE in the database. Two concurrent requests can derive the same slug and both succeed. This is accepted and out of scope.

SLUG TASKS	pytest
1 T-B1 - test_text_norm.py	
2 T-B2 - -k derives	
3 T-B3 - -k suffix	
4 T-B4 - -k alphanumerics	
5 T-B5 - -k explicit	
6 T-B12 - -k blank_description	
√ each: red first, then green	

TPC

SECTION 2 · SUBSECTION 2.1
TOPIC CATALOG ADMINISTRATION (CRUD)

2.1.6 PACKAGE 1 · TREE, WRITE AND DELETE CONTRACT.

TopicNode gains is_active, description, display_order (T-B6) and child_count (T-B7), so the UI can dim inactive rows and preload the edit form. PUT, DELETE, authentication and cache invalidation are fixed by characterization tests that are born green. DELETE keeps returning 204 whether it deactivates or erases (figure 2.1-5).

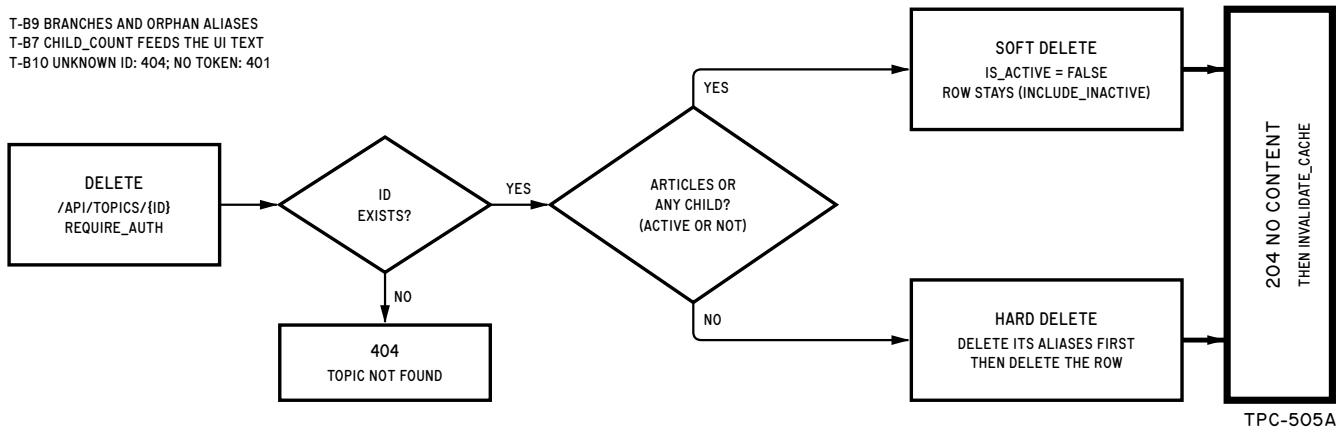


Figure 2.1-5. DELETE Decision in delete_topic

2.1.6.1 Delete Rule.

A topic with articles, or with any child including inactive ones, is only deactivated. An empty topic is erased together with its aliases, because the SQLite tests do not enforce ON DELETE CASCADE (T-B9).

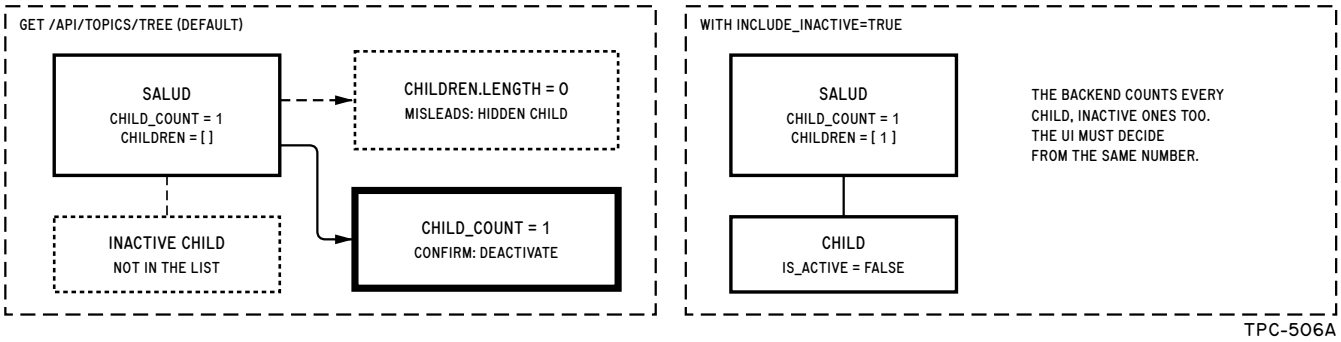


Figure 2.1-6. Hidden Child Case

2.1.7 BACKEND TASKS T-B6 TO T-B13.

Table 2.1-3. Contract Fixed by Each Task

Task	Contract
T-B6	TopicNode: is_active, description, display_order
T-B7	child_count counts inactive children too
T-B8	PUT: rename, redescribe, reactivate; 422, 409, 404
T-B9	DELETE: soft or hard; no orphan aliases
T-B10	unknown parent 404; no token 401
T-B11	invalidate_cache() once per success
T-B13	gate: five-path suite, then full pytest

CAUTION

SQLite runs without PRAGMA foreign_keys=ON, so aliases of an erased topic must be deleted explicitly in delete_topic.

GATE T-B13	pytest

1 PYTEST – the five package paths
  ✓ all green
2 PYTEST – -q, full suite
  ✓ all green
  failures from P1 stay with P1
3 CHECKBOX – tick T-B1 to T-B13

TPC

SECTION 2 · SUBSECTION 2.1
TOPIC CATALOG ADMINISTRATION (CRUD)

2.1.8 PACKAGES 2 AND 3 · FRONTEND.

Package 2 carries the contract to the browser: regenerated api-types.ts, the createTopic, updateTopic and deleteTopic client, flattenTopicTree with the new fields and three mutation hooks that invalidate topicKeys.all. Package 3 builds NewTopicForm and TopicRowCells in src/components/topics/. TopicRowCells moves between the modes of figure 2.1-7; task order is in figure 2.1-8.

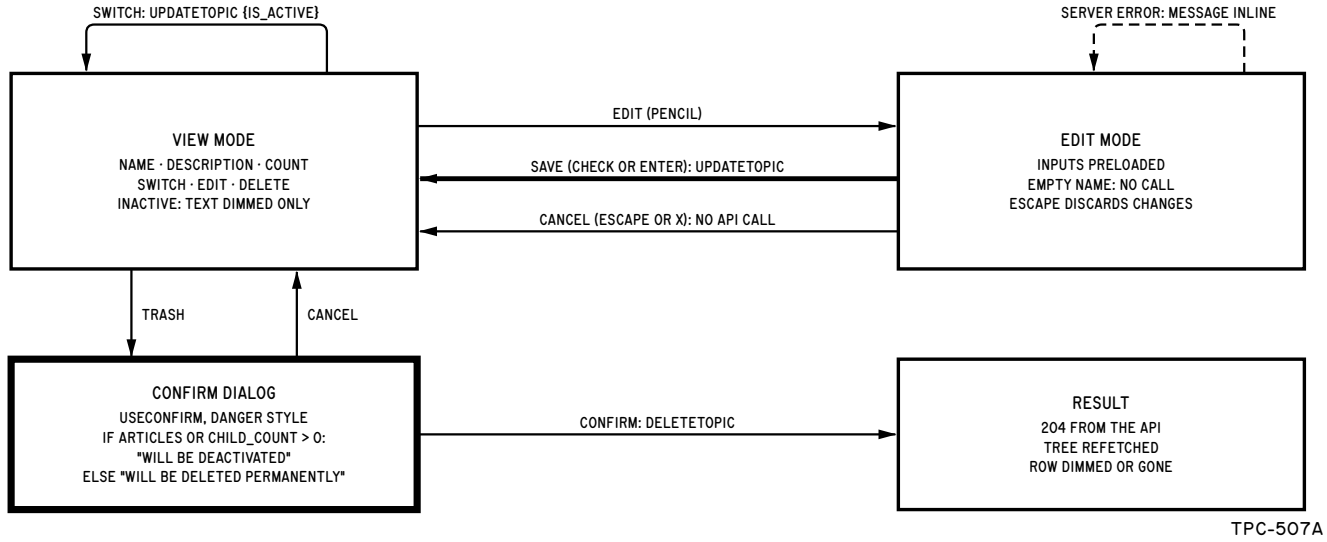


Figure 2.1-7. TopicRowCells Modes and Transitions

2.1.8.1 Dimming Rule.

When is_active is false the text cells carry opacity-45 and the action cells do not, so a dimmed row stays operable. The switch is role="switch" with aria-checked.

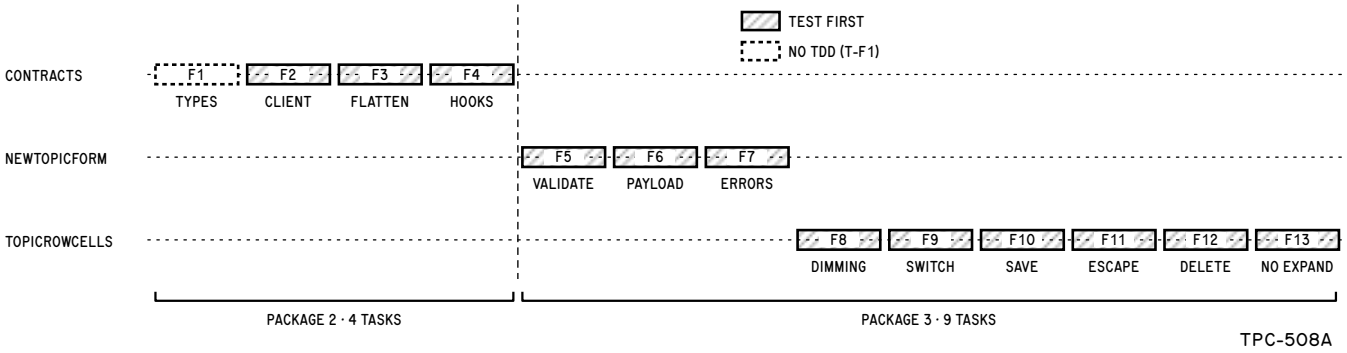


Figure 2.1-8. Frontend Task Order by Lane

2.1.9 DESIGN GUARDRAILS.

Table 2.1-4. Rules Enforced by the Repo Tests

Item	Rule
Inputs, buttons	rounded-lg
Chips	rounded-md
Cards	rounded-xl, border, shadow-sm
Badge	rounded-full
Switch on / off	--pos / --surface-3
Danger	--neg
Forbidden	literal hex, rgb, glass, blur
Guard tests	4, enforced by T-F16

CAUTION

Each action calls e.stopPropagation(); otherwise a click expands the ExpandableRow. api-types.ts is generated: never edit it by hand.

TYPES T-F1 **DEV**

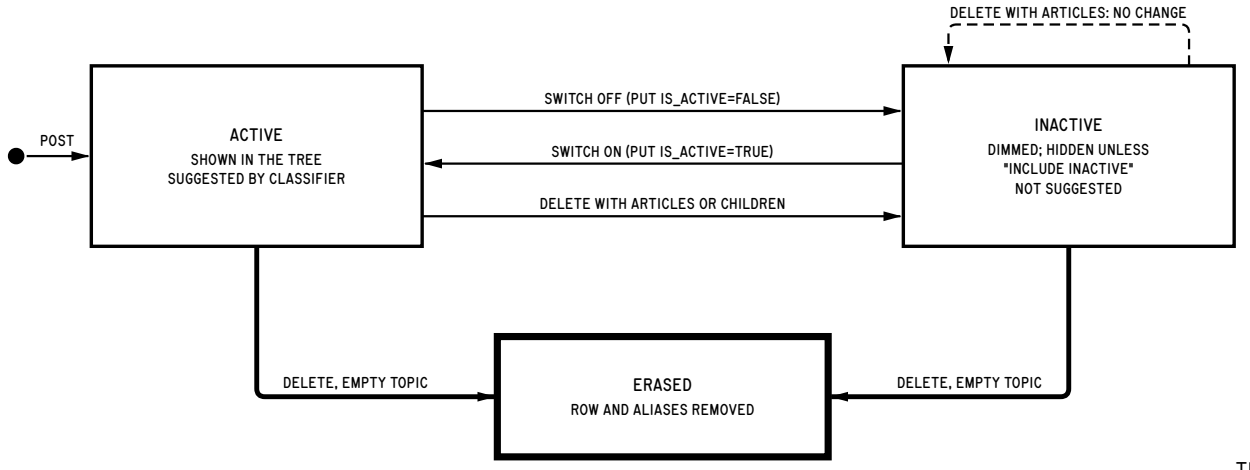
```
1 NPM - run generate:types
  √ no network: tell the user
2 GREP - child_count, api-types
3 TSC - npx tsc --noEmit
  √ consumer errors are fixed
  in T-F3
4 api-types.ts - no hand edits
```

TPC

SECTION 2 · SUBSECTION 2.1
TOPIC CATALOG ADMINISTRATION (CRUD)

2.1.10 PACKAGE 4 · INTEGRATION AND CLOSE.

TopicsPage gets four columns (Tema, Artículos, Estado, actions) with NewTopicForm above the table. Search, "Incluir inactivos", the skeleton, the error Alert and the article panel stay. With no topics and no search, "Crea el primer tema" opens the form (T-F14, T-F15). A topic then follows the lifecycle of figure 2.1-9.



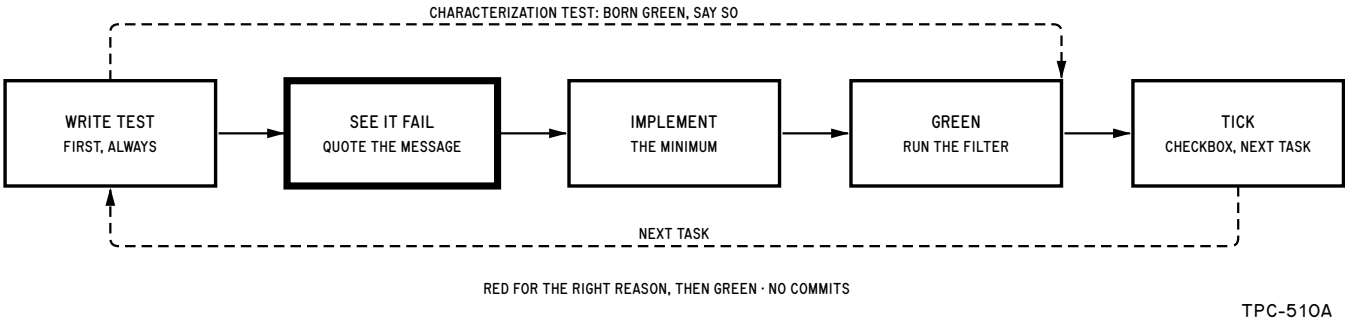
TPC-509A

Figure 2.1-9. Topic Lifecycle Under CRUD

TPC

2.1.10.1 Close.

T-F16 runs vitest, tsc and lint. T-E1 walks the flow in a browser with LocalAnalyzer, never --analyzer gemini. T-E2 records the decision in the Obsidian vault.



TPC-510A

Figure 2.1-10. Red-Green Cycle of One Task

2.1.11 TRACEABILITY.

Table 2.1-5. Requirement to Task

Requirement	Tasks
List as tree with state	T-B6 T-F3 T-F14
Create, optional parent	T-B2 T-F5 T-F6
Unique generated slug	T-B1 T-B3 T-B4 T-B5
Edit name, description	T-B8 T-B12 T-F10 T-F11
Activate, deactivate	T-B8 T-F9
Delete, soft or hard	T-B7 T-B9 T-F12
Auth on mutations	T-B10
Classifier follows edits	T-B11 T-E1

NOTE

Stage 2: alias UI, moving a parent, changing a slug, bulk import, admin-only writes (to confirm), cascading activation, DB-level slug uniqueness.

E2E T-E1	DEV
1 CREATE - root, then child	
2 SWITCH - off, then on	
✓ dimmed; hidden w/o inactive	
3 TRASH - empty topic	
✓ row disappears	
4 TRASH - topic with articles	
✓ stays, inactive	
5 VIEW - light, dark, mobile	